

Linux 内核并发检测机制调研

1. 调研范围

本次一共研究三个机制：

1. lockdep：运行时锁依赖关系校验器，重点检测潜在环形加锁依赖。
2. KCSAN：Kernel Concurrency Sanitizer，重点检测数据竞争。
3. CONFIG_DEBUG_ATOMIC_SLEEP：检测在原子上下文、关中断上下文、显式不可阻塞区间里调用可能睡眠的函数。

这三个机制恰好覆盖三类并发错误：锁顺序错误、共享内存访问错误、上下文语义错误。它们都是在运行时生效的检测机制，一旦观察到危险模式，就在内核真正卡死或破坏数据之前把现场打印到 dmesg。

2. lockdep：给锁顺序建一张有向图

2.1 解决的问题

lockdep 面向的是锁依赖错误，尤其是 ABBA 这类环形等待。最典型的情况是：

```
CPU0 / 线程 A: 先拿 lockA, 再拿 lockB
CPU1 / 线程 B: 先拿 lockB, 再拿 lockA
```

如果两条路径刚好同时执行，就可能变成 A 持有 lockA 等待 lockB，B 持有 lockB 等待 lockA。这恰好对应课程中 Coffman 死锁条件的第四条——循环等待。真正的死锁也许要很久才撞上，但 lockdep 不等到那一刻：只要内核曾经观察到 A -> B 的加锁顺序，又在之后观察到 B -> A 的加锁顺序，就说明系统里已经存在一个潜在的环，此时应该立刻发出警报。

2.2 锁按 class 归类，而非按地址

lockdep 不直接把每一个锁对象都当成图上的节点，因为真实内核里同一处代码可能创建很多同类锁——比如每个 inode 都有自己的锁。如果直接按地址建图，图会非常大，而且很多节点语义相同。

lockdep 使用 lock_class 表示同一种锁。相关结构在 lockdep_types.h：

```
struct lock_class {
    struct hlist_node    hash_entry;    // 哈希表节点，用于快速查找 lock_class
    struct list_head     lock_entry;    // 链入全局 lock_class 列表

    /*
     * These fields represent a directed graph of lock dependencies,
     * to every node we attach a list of "forward" and a list of
     * "backward" graph nodes.
     */
    struct list_head     locks_after, locks_before;
                                // locks_after: 此锁类之后可以拿哪些锁（出边）
                                // locks_before: 哪些锁类在此锁类之前被拿（入边）

    const struct lockdep_subclass_key *key; // 锁类的唯一标识键
    lock_cmp_fn          cmp_fn;          // 锁比较函数
}
```

```

lock_print_fn      print_fn;          // 锁打印函数

unsigned int       subclass;           // 子类编号，用于区分同一锁的不同用法
unsigned int       dep_gen_id;         // 依赖生成 ID，用于增量验证
...
const char        *name;               // 锁类的可读名称
...
};

```

最关键的字段是两条链表：

- `locks_after`：从当前锁类出发，后面可以拿哪些锁，即图里的出边。
- `locks_before`：有哪些锁类会在当前锁类之前被拿，即图里的入边。

在 `kernel/locking/lockdep.c` 里，内核用静态数组保存这些节点和边：

```

static struct lock_list list_entries[MAX_LOCKDEP_ENTRIES]; // 图中的边（依赖关系）
...
struct lock_class lock_classes[MAX_LOCKDEP_KEYS];          // 图中的节点（锁类）
...
struct lock_chain lock_chains[MAX_LOCKDEP_CHAINS];          // 已验证的锁链缓存

```

锁类节点放在 `lock_classes[]`，依赖边放在 `list_entries[]`，已经见过的锁链缓存放在 `lock_chains[]`。内核的调试机制尽量使用静态分配，避免在运行调试代码时引入额外的不确定性。

2.3 每个任务的持锁栈

lockdep 不仅需要全局图，还需要知道当前线程已经持有哪些锁。这个持锁栈中的元素是 `struct held_lock`：

```

struct held_lock {
    /*
     * One-way hash of the dependency chain up to this point. we
     * hash the hashes step by step as the dependency chain grows.
     */
    u64          prev_chain_key; // 当前锁链的哈希前缀，用于依赖链缓存
    unsigned long acquire_ip;    // 获取锁时的指令地址（用于报告）
    struct lockdep_map *instance; // 指向实际锁对象对应的 lockdep_map
    struct lockdep_map *nest_lock; // 嵌套锁（如内部锁）
    ...
    unsigned int  class_idx:MAX_LOCKDEP_KEYS_BITS;
                                     // 此运行时锁实例属于哪个 lock_class
    ...
    unsigned int  irq_context:2;    // 获取锁时的中断上下文类型
    unsigned int  trylock:1;        // 是否为 trylock 获取
    unsigned int  read:2;           // 0=写锁，1=读锁，2=递归读锁
    unsigned int  check:1;          // 是否对此锁执行依赖检查
    unsigned int  hardirqs_off:1;   // 获取锁时硬中断是否已关闭
    unsigned int  sync:1;           // 是否为同步获取
    unsigned int  references:11;    // 引用计数（用于栈式获取）
    unsigned int  pin_count;        // pin 计数
};

```

它保存了几个重要信息：

- `class_idx`：这个运行时锁实例属于哪个 `lock_class`。
- `instance`：实际锁对象对应的 `lockdep_map`。
- `irq_context` / `hardirqs_off`：拿锁时处在普通进程上下文、软中断上下文还是硬中断上下文。这和课程中讲到的「关中断时不能睡眠」是同一个上下文概念。
- `trylock` / `read`：这次获取是 trylock、读锁还是写锁。
- `prev_chain_key`：当前锁链的哈希前缀，用来做依赖链缓存。

所以 lockdep 的视角不是孤立地看每一把锁，而是看当前任务已经拿了一串锁，现在又要拿一把新锁，这条新顺序会不会破坏全局锁顺序图。

2.4 图边的数据结构

图中的一条依赖边用 `struct lock_list` 表示，定义在 `include/linux/lockdep.h`：

```
struct lock_list {
    struct list_head    entry;           // 链表节点
    struct lock_class    *class;         // 指向起点锁类
    struct lock_class    *links_to;      // 指向终点锁类（依赖方向：class -> links_to）
    const struct lock_trace *trace;      // 依赖产生的调用栈跟踪
    u16                 distance;        // BFS 距离
    u8                   dep;            // 依赖类型
    u8                   only_xr;        // 是否仅跨递归读锁

    /*
     * The parent field is used to implement breadth-first search, and the
     * bit 0 is reused to indicate if the lock has been accessed in BFS.
     */
    struct lock_list    *parent;         // BFS 搜索时指向前驱节点，用于路径回溯
};
```

这里的 `parent` 字段是给 BFS 搜索用的。lockdep 报告循环依赖时需要打印已有依赖链的完整路径，BFS 过程中就靠 `parent` 反向还原从起点到目标点的完整锁获取链。

2.5 每次 lock_acquire 都会进入 __lock_acquire

各种锁原语最终会调用 lockdep 的 acquire 路径。入口声明在 `lockdep.h`：

```
extern void lock_acquire(struct lockdep_map *lock, unsigned int subclass,
                        int trylock, int read, int check,
                        struct lockdep_map *nest_lock, unsigned long ip);
```

核心实现是 `lockdep.c` 的 `__lock_acquire()`：

```
/*
 * This gets called for every mutex_lock*()/spin_lock*() operation.
 * We maintain the dependency maps and validate the locking attempt:
 */
static int __lock_acquire(struct lockdep_map *lock, unsigned int subclass,
```

```

        int trylock, int read, int check, int hardirqs_off,
        struct lockdep_map *nest_lock, unsigned long ip,
        int references, int pin_count, int sync)
{
    struct task_struct *curr = current;           // 当前任务
    struct lock_class *class = NULL;
    struct held_lock *hlock;
    unsigned int depth;
    int chain_head = 0;
    int class_idx;
    u64 chain_key;
    ...
    depth = curr->lockdep_depth;                   // 当前任务已持有的锁数量
    ...
    hlock = curr->held_locks + depth;              // 在持锁栈末尾分配位置
    hlock->class_idx = class_idx;                  // 记录锁类
    hlock->acquire_ip = ip;                        // 记录获取位置
    hlock->instance = lock;                        // 记录锁实例
    hlock->nest_lock = nest_lock;
    hlock->irq_context = task_irq_context(curr); // 记录 IRQ 上下文
    hlock->trylock = trylock;
    hlock->read = read;
    hlock->check = check;
    hlock->sync = !!sync;
    hlock->hardirqs_off = !!hardirqs_off;
    ...
    hlock->prev_chain_key = chain_key;             // 继承当前的链哈希
    ...
    chain_key = iterate_chain_key(chain_key, hlock_id(hlock));
                                                    // 将新锁纳入链哈希计算
    ...
    if (!validate_chain(curr, hlock, chain_head, chain_key))
        return 0;                                // 验证失败：发现环或其他违规
    ...
    curr->curr_chain_key = chain_key;              // 更新当前任务的链哈希
    curr->lockdep_depth++;                         // 持锁深度+1，正式纳入持锁栈
    ...
}

```

这个流程的步骤是：先找到这把锁属于哪个 `lock_class`，把这次获取记录成一个新的 `held_lock` 放到当前任务的 `held_locks` 数组末尾，基于已有锁链哈希算出新的 `chain_key`，然后调用 `validate_chain()` 检查这条锁链是否合法，检查通过后才真正增加 `curr->lockdep_depth`，把新锁纳入当前持锁栈。

2.6 锁链缓存

内核拿锁非常频繁，如果每一次加锁都全图 BFS，调试版内核会慢到不可接受。lockdep 因此维护了 `lock_chain` 缓存：

```

struct lock_chain {
    unsigned int      irq_context : 2, // 中断上下文
                    depth      : 6, // 锁链深度
                    base       : 24; // 在 chain_hlocks 数组中的起始索引
    struct hlist_node entry;          // 哈希表节点
    u64               chain_key;      // 锁链的唯一哈希键
};

```

`validate_chain()` 的关键逻辑:

```

static int validate_chain(struct task_struct *curr,
                        struct held_lock *hlock,
                        int chain_head, u64 chain_key)
{
    /*
     * We look up the chain_key and do the O(N^2) check and update of
     * the dependencies only if this is a new dependency chain.
     * 仅当 chain_key 未命中缓存（即这是一条从未见过的锁链）时，才执行昂贵的
     * 依赖检查（check_deadlock）和图更新（check_prevs_add）。
     */
    if (!hlock->trylock && hlock->check &&
        lookup_chain_cache_add(curr, hlock, chain_key)) {
        int ret = check_deadlock(curr, hlock); // 检查是否与已持锁形成死锁

        if (!ret)
            return 0; // 发现死锁，拒绝加锁

        if (!chain_head && ret != 2) {
            if (!check_prevs_add(curr, hlock)) // 为每个已持有锁建立 prev->hlock 依赖边
                return 0;
        }

        graph_unlock();
    }
    ...
    return 1; // 验证通过
}

```

它的策略是：只有 `chain_key` 没见过时，才执行昂贵的依赖检查和图更新。如果这条锁链之前已经被验证过，后面重复出现时可以直接走缓存。

2.7 加边之前先判环

真正的死锁预警发生在 `check_prev_add()`。当当前任务已经持有 `prev`，现在又要拿 `next` 时，lockdep 准备把 `prev -> next` 加入锁依赖图。但在加边之前，它先问：图里是否已经存在 `next -> ... -> prev` 的路径？如果存在，那么再加上 `prev -> next` 就形成了环。这正是课程中 Coffman 第四条件「循环等待」的自动化检测。

源码在 `lockdep.c`：

```

/*
 * Prove that the new <prev> -> <next> dependency would not
 * create a circular dependency in the graph. (We do this by
 * a breadth-first search into the graph starting at <next>,
 * and check whether we can reach <prev>.)
 * 从 next 出发做 BFS，检查是否能到达 prev——若能，则 prev->next 会形成环
 */
ret = check_noncircular(next, prev, trace);
if (unlikely(bfs_error(ret) || ret == BFS_RMATCH))
    return 0; // 检测到环，拒绝添加这条依赖边

```

检查通过之后，才把边加入两边链表：

```

ret = add_lock_to_list(hlock_class(next), hlock_class(prev),
    &hlock_class(prev)->locks_after, distance,
    calc_dep(prev, next), *trace); // prev 的出边: prev -> next
...
ret = add_lock_to_list(hlock_class(prev), hlock_class(next),
    &hlock_class(next)->locks_before, distance,
    calc_depb(prev, next), *trace); // next 的入边: prev -> next

```

一条新依赖被双向登记：在 `prev->locks_after` 里登记 prev 后面可以到 next，在 `next->locks_before` 里登记 next 前面可以来自 prev。这样后续既能向前搜索也能向后搜索。

2.8 BFS 搜索

BFS 的通用实现是 `__bfs()`：

```

static enum bfs_result __bfs(struct lock_list *source_entry,
    void *data,
    bool (*match)(struct lock_list *entry, void *data),
    bool (*skip)(struct lock_list *entry, void *data),
    struct lock_list **target_entry,
    int offset)
{
    struct circular_queue *cq = &lock_cq; // 使用固定大小的环形队列，避免动态分配
    struct lock_list *lock = NULL;
    struct lock_list *entry;
    struct list_head *head;
    ...
    __cq_init(cq);
    __cq_enqueue(cq, source_entry); // 起点入队

    while ((lock = __bfs_next(lock, offset)) || (lock = __cq_dequeue(cq))) {
        if (!lock->class)
            return BFS_EINVALIDNODE;

        if (lock_accessed(lock)) // 已访问过，跳过
            continue;
        else
            mark_lock_accessed(lock); // 标记为已访问
    }
}

```

```

...
if (match(lock, data)) {                // 找到目标
    *target_entry = lock;
    return BFS_RMATCH;
}
...
head = get_dep_list(lock, offset);      // 获取邻接边列表
list_for_each_entry_rcu(entry, head, entry) {
    visit_lock_entry(entry, lock);      // 设置 parent 指针供回溯
    ...
    if (__cq_enqueue(cq, entry))        // 邻居入队
        return BFS_EQUEUEFULL;
}
}

return BFS_RNOMATCH;                    // 未找到目标，无环
}

```

当发现环时，报告内容非常直白：

```

pr_warn("WARNING: possible circular locking dependency detected\n");
...
pr_warn("%s/%d is trying to acquire lock:\n",
        curr->comm, task_pid_nr(curr));
print_lock(check_src);

pr_warn("\nbut task is already holding lock:\n");

print_lock(check_tgt);
pr_warn("\nwhich lock already depends on the new lock.\n\n");

```

这就是 README 里说的还没真正死锁就发出警报。lockdep 不是在等两个线程真的互相阻塞，而是在维护一张全局锁顺序图，只要新的边会让图出现环，它就能立即指出这套加锁顺序存在潜在死锁。

2.9 总结

lockdep 的核心要素是：当前任务持锁栈 + 全局锁类有向图 + BFS 判环 + 锁链缓存。它用于处理锁顺序不一致的问题，即内核中所有锁的获取顺序能不能排成一张无环图。按照我的理解，它是对 Coffman 条件中第四条（循环等待）的自动化监控：在每一次加锁操作时实时检查是否正在创造一条 `prev -> next` 的边，而这条边的加入是否会让已有的有向图中出现从 `next` 回到 `prev` 的路径。

3. KCSAN：用采样 watchpoint 抓数据竞争

3.1 解决的问题

KCSAN 的目标是检测多个执行体并发访问同一内存位置，并且至少一方是写，而且没有被正确同步。例如：

```

/* CPU0 */
global_counter++;

/* CPU1 */
if (global_counter == 0)
    ...

```

如果这两个访问没有锁、没有原子操作、没有 READ_ONCE/WRITE_ONCE 等内核认可的标注，就可能是数据竞争。它不一定会立刻导致系统崩溃，但可能导致读到撕裂值、旧值，或者让编译器和 CPU 重排带来更隐蔽的错误。

按课上的内容看，KCSAN 就是在寻找进入了不安全区域（unsafe region）的轨迹——两个线程对同一内存位置的访问在时间上重叠了，且至少一个是写，且没有适当的同步原语保护。KCSAN 通过采样的方式，在实际执行中探测这些不安全区域是否存在。

3.2 KCSAN 关注内存访问事件

KCSAN 的访问类型定义在 `kcsan-checks.h`：

```

/* Access types -- if KCSAN_ACCESS_WRITE is not set, the access is a read. */
#define KCSAN_ACCESS_WRITE (1 << 0) /* 写访问 */
#define KCSAN_ACCESS_COMPOUND (1 << 1) /* 复合读写（如 i++），需要特殊插桩 */
#define KCSAN_ACCESS_ATOMIC (1 << 2) /* 原子访问，不需要检查 */
/* The following are special, and never due to compiler instrumentation. */
#define KCSAN_ACCESS_ASSERT (1 << 3) /* 断言访问：此处理应是独占的 */
#define KCSAN_ACCESS_SCOPED (1 << 4) /* 作用域访问 */

void __kcsan_check_access(const volatile void *ptr, size_t size, int type);

```

KCSAN 记录的是访问地址 `ptr`、访问大小 `size` 和访问类型（读、写、复合读写、原子访问、断言访问等）。

真正导出的入口很小，在 `core.c`：

```

void __kcsan_check_access(const volatile void *ptr, size_t size, int type)
{
    check_access(ptr, size, type, _RET_IP_); // 进入核心检查逻辑
}
EXPORT_SYMBOL(__kcsan_check_access);

```

内核注释说明，KCSAN 使用编译器给 ThreadSanitizer 生成的同类插桩：

```

/*
 * KCSAN uses the same instrumentation that is emitted by supported compilers
 * for ThreadSanitizer (TSAN).
 *
 * When enabled, the compiler emits instrumentation calls (the functions
 * prefixed with "__tsan" below) for all loads and stores that it generated;
 * inline asm is not instrumented.
 */

```

也就是说 KCSAN 能工作，是因为普通内存读写在编译后会多一层检查入口。

3.3 每个 CPU / 任务上下文都有 KCSAN 状态

KCSAN 需要知道当前上下文是否临时关闭检测、是否处于原子访问区域、是否有 scoped access 等。状态结构在 `kcsan.h`：

```
struct kcsan_ctx {
    int disable_count;           /* 禁用计数器, >0 时 KCSAN 不检查 */
    int disable_scoped;         /* 作用域禁用计数器 */
    int atomic_next;            /* 接下来几个访问视为原子操作 */
    ...
    int atomic_nest_count;      /* 原子嵌套深度 */
    bool in_flat_atomic;        /* 是否处于平坦原子区域 */

    /*
     * Access mask for all accesses if non-zero.
     */
    unsigned long access_mask;   /* 非零时限制检查的访问位 */

    /* List of scoped accesses; likely to be empty. */
    struct list_head scoped_accesses;
    ...
};
```

`disable_count` 可以临时关闭 KCSAN, `atomic_nest_count` / `in_flat_atomic` 用来表达这一段访问按原子区域处理, `access_mask` 用于位级别检查。

3.4 快路径：检查 watchpoint

KCSAN 的核心函数是 `check_access()`：

```
static __always_inline void
check_access(const volatile void *ptr, size_t size, int type, unsigned long ip)
{
    atomic_long_t *watchpoint;
    long encoded_watchpoint;
    ...
again:
    watchpoint = find_watchpoint((unsigned long)ptr, size,
                                !(type & KCSAN_ACCESS_WRITE),
                                &encoded_watchpoint);
    // 查找是否有其他 CPU 在此地址设置了 watchpoint

    ...
    if (unlikely(watchpoint != NULL))
        kcsan_found_watchpoint(ptr, size, type, ip, watchpoint, encoded_watchpoint);
    // 撞上别人的 watchpoint: 发现潜在竞争!

    else {
        struct kcsan_ctx *ctx = get_ctx();

        if (unlikely(should_watch(ctx, ptr, size, type))) {
            kcsan_setup_watchpoint(ptr, size, type, ip);
            return; // 采样决定设置 watchpoint, 进入慢路径
        }
    }
}
```

```

    ...
}
}

```

可以把它拆成两步：先看看当前访问的地址范围是否撞上了已有 watchpoint。如果撞上，说明另一个 CPU 或线程正在盯着这块内存，当前访问可能与它竞争；如果没有撞上，按采样策略决定自己要不要设置一个 watchpoint。

这就是 KCSAN 和传统全量记录所有访问的 sanitizer 不一样的地方：KCSAN 是采样式的。大多数访问走很轻的快路径，只有少数访问会被选中进入慢路径。

3.5 慢路径：设置 watchpoint 并人为扩大竞争窗口

当 `should_watch()` 决定观察本次访问时，会进入 `kcsan_setup_watchpoint()`：

```

static ninline void
kcsan_setup_watchpoint(const volatile void *ptr, size_t size, int type, unsigned long ip)
{
    const bool is_write = (type & KCSAN_ACCESS_WRITE) != 0;
    const bool is_assert = (type & KCSAN_ACCESS_ASSERT) != 0;
    atomic_long_t *watchpoint;
    u64 old, new, diff;
    ...
    watchpoint = insert_watchpoint((unsigned long)ptr, size, is_write);
                                // 在全局 watchpoint 表中注册此地址
    if (watchpoint == NULL) {
        atomic_long_inc(&kcsan_counters[KCSAN_COUNTER_NO_CAPACITY]);
        goto out_unlock;        // watchpoint 表已满
    }
    ...
    old = is_reorder_access ? 0 : read_instrumented_memory(ptr, size);
                                // 记录当前内存值

    /*
     * Delay this thread, to increase probability of observing a racy
     * conflicting access.
     */
    delay_access(type);          // 人为延迟，拉宽竞态窗口

    if (!is_reorder_access) {
        new = read_instrumented_memory(ptr, size);
    } else {                    // 读取延迟后的内存值
        new = 0;
        access_mask = 0;
    }
    ...
    if (!consume_watchpoint(watchpoint)) {
        ...                    // watchpoint 未被其他 CPU 消费：可能是值变化竞争
        kcsan_report_known_origin(ptr, size, type, ip,
                                   value_change, watchpoint - watchpoints,
                                   old, new, access_mask);
    } else if (value_change == KCSAN_VALUE_CHANGE_TRUE) {
        ...                    // 值变化了但无已知来源
        kcsan_report_unknown_origin(ptr, size, type, ip,

```

```

        old, new, access_mask);
    }
    ...
    remove_watchpoint/watchpoint); // 清理 watchpoint
}

```

这里三个关键操作：

1. `insert_watchpoint()`：把当前访问的地址、大小、读写类型登记到全局 watchpoint 表。
2. `delay_access(type)`：主动延迟当前执行体，让其他 CPU 更有机会在这段时间里访问同一地址。
3. `consume_watchpoint()` / value-change 检查：判断这段时间里是否有其他冲突访问撞上，或者值是否发生了意料之外的变化。

KCSAN 的基本策略不是穷举所有可能的交错，而是挑一个访问停下来，把原本很窄的竞态窗口拉宽，借以探测是否有其他执行体在同一地址上产生冲突访问。这和PPT中「不安全区域」的概念完全对应：KCSAN 通过人为延迟把原本可能擦肩而过的两个访问强行拉入重叠区域，从而暴露潜在的数据竞争。

3.6 发现 watchpoint 后的行动

当当前访问撞上别人设置的 watchpoint，会进入 `kcsan_found_watchpoint()`：

```

static noline void kcsan_found_watchpoint(const volatile void *ptr,
                                           size_t size,
                                           int type,
                                           unsigned long ip,
                                           atomic_long_t *watchpoint,
                                           long encoded_watchpoint)
{
    const bool is_assert = (type & KCSAN_ACCESS_ASSERT) != 0;
    struct kcsan_ctx *ctx = get_ctx();
    ...
    if (!kcsan_is_enabled(ctx))
        return;
    ...
    consumed = try_consume_watchpoint/watchpoint, encoded_watchpoint);
                // 尝试消费 watchpoint (双方各检测到对方)
    ...
    if (consumed) {
        kcsan_save_irqtrace(current);
        kcsan_report_set_info(ptr, size, type, ip, watchpoint - watchpoints);
        kcsan_restore_irqtrace(current);
    } else {
        atomic_long_inc(&kcsan_counters[KCSAN_COUNTER_REPORT_RACES]);
    }

    if (is_assert)
        atomic_long_inc(&kcsan_counters[KCSAN_COUNTER_ASSERT_FAILURES]);
    else
        atomic_long_inc(&kcsan_counters[KCSAN_COUNTER_DATA_RACES]);
    ...
}

```

报告函数在 `report.c`。报告类型也很清楚：

```
static const char *get_bug_type(int type)
{
    return (type & KCSAN_ACCESS_ASSERT) != 0 ? "assert: race" : "data-race";
}
```

普通竞争打印为 data-race，显式断言失败打印为 assert: race。

3.7 标注机制

内核里并非所有无锁访问都错误，有些访问是刻意的，比如统计计数、RCU 读侧、只要求最终一致的状态位等。KCSAN 因此提供了标注机制：用原子操作或 `READ_ONCE()` / `WRITE_ONCE()` 表示这类访问有明确语义；用 `data_race()` 表示这里确实可能竞争，但这是作者有意接受的竞争；反过来用 `ASSERT_EXCLUSIVE_WRITER()` / `ASSERT_EXCLUSIVE_ACCESS()` 声明这里理论上应该没有并发写或并发访问，如果有就报告。

相关宏在 `kcsan-checks.h` 中：

```
#define ASSERT_EXCLUSIVE_WRITER(var) \
    __kcsan_check_access(&(var), sizeof(var), KCSAN_ACCESS_ASSERT) \
    // 断言：此处应无并发写者
...
#define ASSERT_EXCLUSIVE_ACCESS(var) \
    __kcsan_check_access(&(var), sizeof(var), \
        KCSAN_ACCESS_WRITE | KCSAN_ACCESS_ASSERT) \
    // 断言：此处应无任何并发访问
```

这说明 KCSAN 不只是被动抓 race，还允许内核开发者把「这里必须独占」的设计意图写进代码，设计意图一旦在运行时被破坏就会发出警报。

3.8 总结

KCSAN 的核心要素是：编译器内存访问插桩 + 采样 watchpoint + 人为延迟扩大窗口 + 冲突访问/值变化报告。它抓的是共享内存层面的数据竞争，不需要知道你用的是 mutex、spinlock 还是 RCU，只要两个访问真的落到同一片内存而且访问类型构成冲突，它就有机会报告。但因为它是采样式的，一次运行没报不等于绝对没有 race。从 PPT 的内容上看，KCSAN 是进度图中不安全区域的运行时探测器：它通过采样和延迟，在实际执行轨迹中寻找两个线程的临界区（对同一内存位置的读写）是否发生了非预期的重叠。

4. CONFIG_DEBUG_ATOMIC_SLEEP：高级睡眠检查

4.1 解决的问题

用户态程序里，线程拿着普通 mutex 时阻塞等待通常是合法的。但内核里有很多上下文绝对不能睡眠：持有自旋锁时、关中断时、位于硬中断或软中断相关路径时、显式声明为不可阻塞的区间里。如果这些地方调用了可能睡眠的函数——比如内存分配、mutex_lock、wait_event 等——就可能把内核带进非常危险的状态。

`CONFIG_DEBUG_ATOMIC_SLEEP` 就是专门检查这类错误的。

这和课程中讲到的锁类型选择密切相关：自旋锁（spinlock）在持有时不休眠、一直占用 CPU 反复检查锁状态，适用场景是锁持有时间极短；互斥锁（mutex）在锁被占用时会让线程休眠，等待被唤醒。如果在自旋锁持有的原子上下文里调用了可能睡眠的函数，就相当于在不该睡的地方睡了一——这会导致调度器在禁止调度的上下文中试图切换任务，引发内核 panic 或难以调试的死锁。

4.2 might_sleep: 可能睡眠函数的自我标注

入口宏在 `kernel.h`:

```
#ifdef CONFIG_DEBUG_ATOMIC_SLEEP
extern void __might_resched(const char *file, int line, unsigned int offsets);
extern void __might_sleep(const char *file, int line);
...
/**
 * might_sleep - annotation for functions that can sleep
 *
 * this macro will print a stack trace if it is executed in an atomic
 * context (spinlock, irq-handler, ...). Additional sections where blocking is
 * not allowed can be annotated with non_block_start() and non_block_end()
 * pairs.
 */
# define might_sleep() \
    do { __might_sleep(__FILE__, __LINE__); might_resched(); } while (0)
        // 在不可睡眠上下文中调用此宏会触发警告
```

可能睡眠的函数自己要声明「我可能睡」。在 debug 配置打开时, `might_sleep()` 会把调用点的文件名和行号传进检查函数; 如果当前上下文不允许睡, 就能把源代码位置打印出来。

4.3 non_block_start / non_block_end: 人工声明不可阻塞区间

除了 preempt count、IRQ 状态这些硬指标, 内核还有一些语义上不能阻塞的区间, 可以通过显式标注来保护:

```
/**
 * non_block_start - annotate the start of section where sleeping is prohibited
 */
# define non_block_start() (current->non_block_count++)
        // 进入不可阻塞区间, 计数器加1

/**
 * non_block_end - annotate the end of section where sleeping is prohibited
 */
# define non_block_end() WARN_ON(current->non_block_count-- == 0)
        // 退出不可阻塞区间, 计数器减1
        // 如果计数器已经为0则触发 WARN
```

计数存在 `task_struct` 里, 见 `sched.h`:

```
#ifdef CONFIG_DEBUG_ATOMIC_SLEEP
    int non_block_count; // >0 表示当前处于禁阻塞区间
#endif
```

禁睡区间是跟当前任务绑定的: 进入一次 `non_block_start()`, `current->non_block_count` 增加; 退出时减少。如果在这个区间里调用 `might_sleep()`, 即使 preempt count 看起来正常也会触发警告。

4.4 preempt_count 与 irqs_disabled

是否处于原子上下文，一个核心指标是 `preempt_count()`。相关宏在 `preempt.h`：

```
#define in_atomic() (preempt_count() != 0)           // 处于原子上下文（禁止抢占）
...
#define preemptible() (preempt_count() == 0 && !irqs_disabled())
// 可抢占：抢占计数为0 且 中断未关闭
```

`preempt_count() != 0` 表示当前处在某种禁止抢占或原子嵌套状态，`irqs_disabled()` 表示当前 CPU 关中断，`preemptible()` 只有 `preempt count` 为 0 且中断没有关闭才算可抢占。这些条件与能否睡眠直接相关：睡眠意味着调度器可能切走当前任务，但如果当前代码处在禁止调度的状态，强行睡眠就破坏了内核的上下文约束。

4.5 __might_resched

`might_sleep()` 调用 `__might_sleep()`，再进入 `__might_resched()`。核心检查在 `core.c`：

```
void __might_sleep(const char *file, int line)
{
    unsigned int state = get_current_state();
    ...
    __might_resched(file, line, 0);
}
EXPORT_SYMBOL(__might_sleep);
```

`__might_resched` 的部分代码：

```
void __might_resched(const char *file, int line, unsigned int offsets)
{
    ...
    rcu_sleep_check();           // 额外的 RCU 睡眠检查

    if ((resched_offsets_ok(offsets) && !irqs_disabled() &&
        !is_idle_task(current) && !current->non_block_count) ||
        system_state == SYSTEM_BOOTING || system_state > SYSTEM_RUNNING ||
        oops_in_progress)
        return;                 // 安全：允许睡眠
    ...
    pr_err("BUG: sleeping function called from invalid context at %s:%d\n",
           file, line);         // 报告：在不该睡的地方调用了可能睡眠的函数
    pr_err("in_atomic(): %d, irqs_disabled(): %d, non_block: %d, pid: %d, name: %s\n",
           in_atomic(), irqs_disabled(), current->non_block_count,
           current->pid, current->comm);
    pr_err("preempt_count: %x, expected: %x\n", preempt_count(),
           offsets & MIGHT_RESCHED_PREEMPT_MASK);
    ...
    debug_show_held_locks(current); // 打印当前持有的锁（与 lockdep 联动）
    if (irqs_disabled())
        print_irqtrace_events(current);
    ...
    dump_stack();               // 打印完整调用栈
```

```
    add_taint(TAINT_WARN, LOCKDEP_STILL_OK);
}
```

这段逻辑相当于一道门禁：preempt/RCU 偏移符合预期、没有关中断、不是 idle task、不在 non_block 区间，就允许，直接 return；否则打印 BUG、上下文状态、当前持锁信息、栈回溯。注意它还会调用 debug_show_held_locks(current)——这和 lockdep 形成了联动，当某个睡眠错误发生时，报告不仅告诉你哪里睡了，还尽量告诉你当时手里握着哪些锁。

4.6 如果真的调度进了 non_block 区间

除了 might_sleep() 主动检查，调度器切换路径也有防线。在 core.c 中：

```
#ifdef CONFIG_DEBUG_ATOMIC_SLEEP
    if (!preempt && READ_ONCE(prev->__state) && prev->non_block_count) {
        printk(KERN_ERR "BUG: scheduling in a non-blocking section: %s/%d/%i\n",
            prev->comm, prev->pid, prev->non_block_count);
        dump_stack();
        add_taint(TAINT_WARN, LOCKDEP_STILL_OK);
    }
    // 任务在 non_block 区间内被调度出去，报警
#endif
```

即使某些路径没有通过 might_sleep() 及时暴露，只要任务真的在 non_block_count > 0 的状态下进入调度，也会得到一条更直接的警报。

4.7 总结

CONFIG_DEBUG_ATOMIC_SLEEP 的核心要素是：可能睡眠函数的 might_sleep 标注 + preempt/IRQ/non_block 状态检查 + 栈与持锁信息报告。它关注上下文语义错误。从课内内容看，这个机制把自旋锁与互斥锁的关键区别（一个不休眠、一个可能休眠）转化成了可自动检查的运行时条件，确保开发者在原子上下文中不会无意中引入睡眠操作。

5. 对三个机制的总结

机制	主要配置项	抓的问题	核心数据/状态	检测方式	典型报告
lockdep	CONFIG_PROVE_LOCKING	锁顺序成环、ABBA、IRQ 上下文锁反转	lock_class、held_lock、lock_list、lock_chain	每次加锁时维护有向图，加边前 BFS 判环	WARNING: possible circular locking dependency detected
KCSAN	CONFIG_KCSAN	共享内存数据竞争	kcsan_ctx、watchpoint、访问类型 bit	编译器插桩普通内存访问，采样设置 watchpoint 并延迟	data-race / assert: race
DEBUG_ATOMIC_SLEEP	CONFIG_DEBUG_ATOMIC_SLEEP	原子/关中断/禁阻塞上下文里调用睡眠函数	preempt_count、irqs_disabled()、current->non_block_count	might_sleep() 和调度路径检查当前上下文	BUG: sleeping function called from invalid context

分开来看，lockdep 关注锁与锁之间的顺序关系，如果顺序图出现环就能在真正卡死前发出警报；KCSAN 关注内存与内存访问之间的并发冲突，只要共享地址上出现未同步冲突就可能报告；DEBUG_ATOMIC_SLEEP 在当前上下文不允许睡眠时，把内核中那些不能阻塞的区域变成可检查的运行时条件。

内核并发检测的价值在于，它把并发程序中最难靠肉眼稳定复现的部分转化成了三类运行时事实：锁顺序事实（我见过哪些锁先后关系）、内存访问事实（谁在什么时候访问了同一片内存）、上下文事实（当前的代码能否睡眠）。